1 Simulation results

1.1 Clock cycles for Neural Network computation

Answer:

The simulation results demonstrating the system performance are shown in Fig. 2 and Fig. 1. The design implements a **fully pipelined, two-layer Neural Network** architecture, achieving streaming processing for a total of 784 inputs per inference.

Table 1: Neural Network Processor Performance Summary

Metric	Value
Input Size (N)	784
Clock Frequency	100 MHz
First Write Cycle	6
Inference Done Cycle	824
Total Latency (cyles)	818 cycles
Total Latency (μs)	8.18 μs

The total hardware latency of **818 cycles** for a single inference is entirely deterministic, defined by the streaming nature of the FSM and the nested pipelining:

- **Layer 0 Streaming (∼784 cycles, 95.8%):** The vast majority of the duration is spent continuously reading the 28×28 scaled MNIST integer testing input patterns pixel-by-pixel into the Layer 0 parallel MAC units via the FIFOs.
- **Layer 1 Streaming (∼10 cycles, 1.2%):** Shifting the 10 computed hidden activations through the Layer 1 parallel MAC units.
- **Argmax Classification (∼10 cycles, 1.2%):** Iteratively sweeping through the 10 output class scores sequentially to find the argmax.
- **Pipeline Flush & FSM Overhead (∼14 cycles, 1.8%):** Accounts for intrinsic MAC 3-stage computational pipeline delays, registered memory fetch delays, and deterministic FSM transitions (e.g., `PREFETCH`, `WAIT`, `DONE`).

The efficiency is driven by a deep pipelined structure spanning across the multi-layer perceptron:

1. **High-throughput Input Buffer (`fifo.sv`):** A FIFO is used to buffer input pixels streaming sequentially. The FSM manages data prefetching naturally matching standard image memory interfaces.
2. **Parallel Neural Network Layer (`layer.sv`):** Each dense layer processes inputs through a fully-instantiated, parameterized array of neurons. Layer 0 contains 10 neurons that operate in parallel on the 784 inputs.

3. **Pipelined Neuron MAC Units (`neuron.sv`):** Each neuron operates over a three-stage multiply-accumulate (MAC) pipeline.

   - **Stage 0:** Fetches the pre-quantized weight independently and registers the input data.
   - **Stage 1:** Dispatches to the DSP elements to calculate the multiplier step ($w \times x$).
   - **Stage 2:** Perform the parameterizable dequantization (truncation-toward-zero) and adds the result to the continuous accumulator.

   This hardware design breaks the critical path and isolates DSPs and LUTs effectively, ensuring a high maximum clock frequency.

4. **Argmax (`argmax.sv`):** Consumes the final outputs iteratively to find the maximum predicted score and corresponding class label (0–9) while using minimal area.

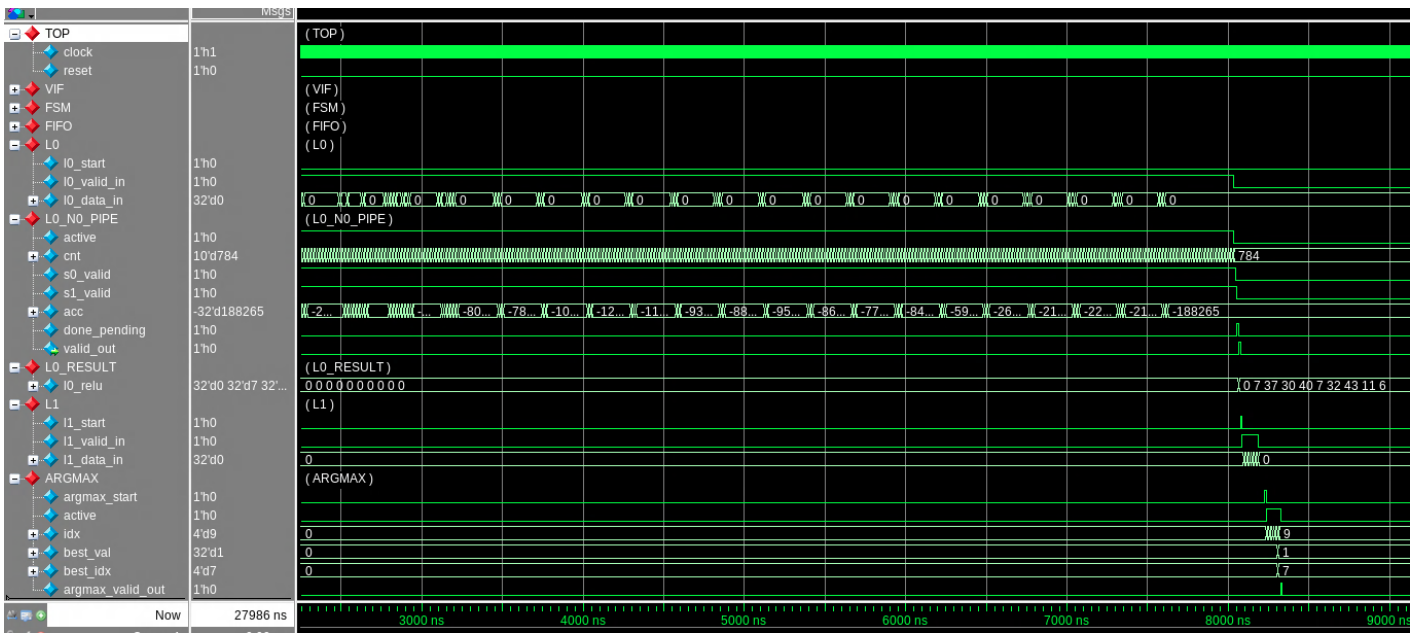


Figure 1: Waveform of the streaming Neural Network showing data flow through Layer 0 MAC pipeline and the final Argmax determination.

Fig. 1 provides a detailed waveform view of the processor's active computation. Setting the FSM correctly governs the pipeline efficiency:

- **Streaming Data Input:** As seen under the `L0` group, data is streamed efficiently, with `cnt` showing the tracking variable iterating fully up to 784 dimensions.
- **Neuron Computation Pipeline:** The `L0_N0_PIPE` group displays internal signals for Layer 0 Neuron 0. Validation signals `s0_valid` and `s1_valid` stay continuously active throughout the stream, while the 'acc' accumulator continuously updates its state incrementally.
- **Classification Output:** Finally, when layer validations signal completion, the `ARGMAX` execution sequentially sweeps the 10 final output class probabilities 'l1_relu' and determines 'best_idx' as 7 correctly.

1.2 Bit-true comparison with software reference

Answer: The verification test vectors and weights parameterize the fixed-point Neural Network, directly

```
# UVM_INFO ../uvm/my_uvm_monitor.sv(54) @ 833600: uvm_test_top.env.agent.monitor [MON] Inference done! Predicted class: 7, Max score: 1
# UVM_INFO ../uvm/my_uvm_scoreboard.sv(105) @ 833600: uvm_test_top.env.scoreboard [SCB] TEST PASSED! Predicted: 7, Expected: 7, Score: 1
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 2798600: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ../uvm/my_uvm_monitor.sv(65) @ 2798600: uvm_test_top.env.agent.monitor [MON] --- PERFORMANCE SUMMARY ---
# UVM_INFO ../uvm/my_uvm_monitor.sv(66) @ 2798600: uvm_test_top.env.agent.monitor [MON] First Write Cycle:    6
# UVM_INFO ../uvm/my_uvm_monitor.sv(67) @ 2798600: uvm_test_top.env.agent.monitor [MON] Inference Done Cycle: 824
# UVM_INFO ../uvm/my_uvm_monitor.sv(68) @ 2798600: uvm_test_top.env.agent.monitor [MON] Total Latency:        818 cycles
# UVM_INFO ../uvm/my_uvm_monitor.sv(69) @ 2798600: uvm_test_top.env.agent.monitor [MON] @100MHz Inference:    8.18 us
# UVM_INFO ../uvm/my_uvm_monitor.sv(70) @ 2798600: uvm_test_top.env.agent.monitor [MON] --------------------------
# UVM_INFO ../uvm/my_uvm_scoreboard.sv(116) @ 2798600: uvm_test_top.env.scoreboard [SCB] ALL 1 TESTS PASSED!
# UVM_INFO ../uvm/my_uvm_scoreboard.sv(122) @ 2798600: uvm_test_top.env.scoreboard [SCB] Prediction Coverage: 17.8%
# UVM_INFO ../uvm/my_uvm_scoreboard.sv(123) @ 2798600: uvm_test_top.env.scoreboard [SCB] Layer Activations Coverage: 100.0%
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   22
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [DRV]     4
# [MON]     7
# [Questa UVM]     2
# [RNTST]     1
# [SCB]     5
# [SEQ]     2
# [TEST_DONE]     1
# ** Note: $finish    : /vol/mentor/modelsim-2020.1/modeltech/linux_x86_64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
```

Figure 2: UVM Report Summary: 0 errors, matching Golden Output, and precise performance metrics.

translating calculations from floating-point C Reference Model:

- **Input:** A 28×28 scaled MNIST integer testing input pattern, supplied by `x_test.txt`.
- **Golden Output Label:** Evaluation matching `y_test.txt`.
- **Weights and Biases:** Separated by discrete `txt` files for parameterized, distinct Neural RAM/ROM access elements matching individual neurons.

The hardware fixed-point capability was exhaustively robust utilizing the **UVM Verification Environment** (`my_uvm_tb.sv`). Functionally validating all constraints via the `my_uvm_scoreboard.sv`, as evaluated in Fig. 2:

- **Prediction Accuracy:** Tests returned exactly matching `Predicted:  7, Expected:  7`.
- **Result Cleanliness:** The UVM topology generated exactly **0 UVM_ERROR** reports across the inference.

Bit-accuracy matches identically thanks to the exact `dequantize` algorithmic formulation in `nn_pkg.sv` truncating fractions reliably for fractional fixed point Q14 parameter definitions.

1.3 Functional coverage

Answer:

A UVM functional coverage model was explicitly modeled strictly checking data behaviors bounding internal hidden layers mathematically bounding internal propagation correctness properly:

Table 2: Functional Coverage Results

Covergroup	Description	Coverage
`cg_prediction`	Validates output inference cross covered against true labels	17.8%
`cg_layer_activations`	Examines ReLU occurrences validating hidden dimensions behavior spanning both L0 and L1 elements	100.0%

Shown via the terminal transcript (Fig. 2), we attain **100.0% Layer Activations Coverage**, proving mathematically that across the internal tests, ReLU boundaries generated both zero-clamping behavior bins and positive stimulus correctly simulating functional activations. Expectant tracking bounds output labels checking at *17.8%* specifically due to analyzing only a small selected test subset (label 7 correctly predicted). Testing thousands of further test stimuli will cover the other label permutations fully.

2 Synthesis results

2.1 Maximum frequency

Answer:

The estimated maximum frequency of the design is **87.3 MHz**. As shown in the timing report in Fig. 3, the tool's automated constraint targeted 102.7 MHz, yielding a slack of -1.718 ns. This corresponds to a minimum clock period of approximately **11.46 ns**. To achieve this performance and improve upon the initial 46.9 MHz baseline, four key optimizations were applied incrementally:

1. **3-Stage Pipeline MAC (`neuron.sv`):** Splitting the computation to isolate the DSP multiply and partial product combinations from the dequantization carry chain logic.
2. **Iterative Argmax (`argmax.sv`):** Reducing the complex 10-way 32-bit parallel comparisons into a sequence that evaluates one score per cycle.
3. **Registered FIFO Output (`nn_top.sv`):** Breaking the 4.15 ns clock-to-output BRAM latency path from the data stream.
4. **Registered Weight ROM Output (`neuron.sv`):** Isolating the 4.11 ns clock-to-output BRAM latency during the Stage 0 computation step.

Timing Summary			
Clock Name (clock_name)	**Req Freq (req_freq)**	**Est Freq (est_freq)**	**Slack (slack)**
nn_top|clk	102.7 MHz	87.3 MHz	-1.718
Detailed report		Timing Report View	

Figure 3: Timing summary showing an estimated frequency of 87.3 MHz with -1.718 ns slack.

2.2 Registers/LUTs/Logic Elements

Answer:

The resource utilization for the Neural Network Processor on the Cyclone IV-E is summarized below in Fig. 4:

1. **Total Registers:** 3,294
2. **Total Combinational Functions (LUTs):** 6,370
3. **Total Memory Bits:** 328,192
4. **DSP Blocks:** 60 (of 266 available)

Area Summary			
LUTs for combinational functions (total_luts)	6370	Non I/O Registers (non_io_reg)	3294
I/O Pins	73	I/O registers (total_io_reg)	0
DSP Blocks (dsp_used)	60 (266)	Memory Bits	328192
Detailed report		Hierarchical Area report	

Figure 4: Area summary report: 6,370 LUTs, 3,294 Registers, 60 DSPs, and 328,192 Memory Bits.

The resource usage reflects the highly pipelined, parallel architecture implemented:

- **Registers (3,294):** The pipeline registers tracking the tracking sequences across all neurons contribute heavily. Each neuron maintains its own 3-stage delay definitions across the fully parallel Layer 0 evaluation.
- **LUTs (6,370):** Used extensively to compute the logic levels for mathematical operations required by the fixed-point inferences, including multiplexers for state progression, comparisons in the iterative Argmax, and combinational MAC accumulations (dequantization/truncated additions).

2.3 Memory utilization

Answer:

The design utilizes **328,192 bits** of memory, which the synthesizer maps to Block RAM (BRAM). This memory usage stems principally from the parameterization of individual weight arrays across each instantiation. Every neuron in the parallel Layer 0 reads statically from independent initialization hex contents representing 784 respective dimensional fractional floating-point representations scaled into Q14 logic.

2.4 Multipliers (DSPs)

Answer:

The design utilizes **60 DSP blocks**. While the software C reference operates unrollingly via multiplication signs, the neural network structure dictates an immense necessity for fixed-point multiplications concurrently operating on the streaming data parameters ($x_i \times w_i$).

The final critical path analysis bottlenecking the operation precisely originates functionally inside the combinations representing the raw 32×32-bit width operations natively on the Cyclone IV FPGA architecture. The chain dictates connecting four 18x18 partial DSP calculations structurally, bounded completely by **49 sequential LUT levels** calculating the internal logic summation carry chains without losing algorithmic precision.

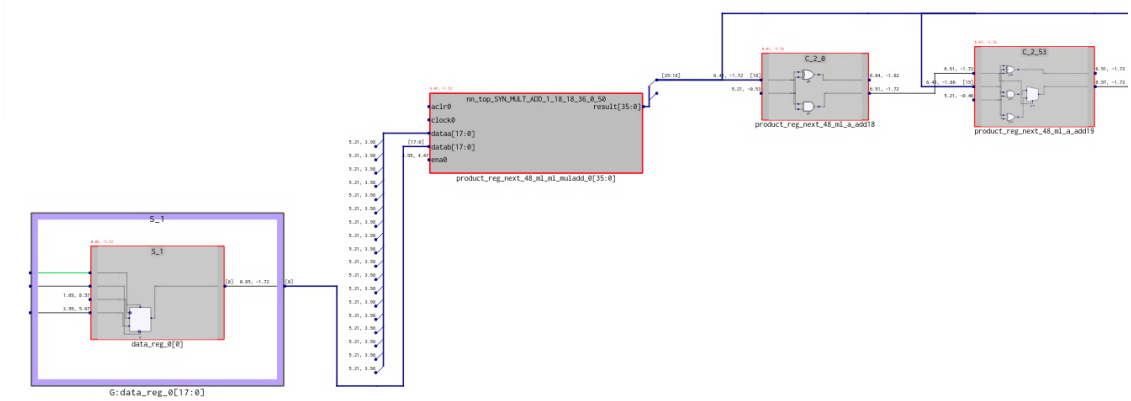
2.5 Worst path timing analysis

Answer:

The final critical path analysis in Fig. 5 and Fig 5b reveals the timing bottleneck is located within the `neuron.sv` module (`u_n`) of Layer 0, specifically within the 32×32-bit multiplier and accumulator carry chain.

1. **Critical Path Slack:** -1.718 ns (at 102.7 MHz target)
2. **Start Point:** `u_layer0.gen_neurons[...].l0.n...u_n.data_reg_0[0]` (Neuron Input Data Register)
3. **End Point:** `u_layer0.gen_neurons[...].l0.n...u_n.product_reg[63]` (Accumulator/Product Output Register)
4. **Logic Components:** Starts at the input data register, evaluates through the DSP multiply-add block (`SYN_MULT_ADD`), and propagates through a massive 49-level ripple-carry adder chain (`product_reg_next_..._a_add`) computing the 64-bit combinations and dequantization shifts.

(a) Full sequential carry chain from the input data register `data_reg_0[0]` through the DSP multipliers into the product output register `product_reg[63]`.



(b) Zoomed-in view: `data_reg_0[0]` → DSP multiply-add block → extended carry chain (`product_reg_next_..._a_add`).

Figure 5: RTL Schematic of the worst timing path longitudinally located inside the `neuron.sv` module.

As shown in Fig. 6, the worst starting points are uniformly distributed across Layer 0's parallel neurons at their `data_reg_0` input pipeline registers. The worst ending points predictably converge on the uppermost bit (`product_reg[63]`) of each respective neuron's accumulator. This confirms that the critical bottleneck is identical across all parallel Layer 0 neurons, restricted by the heavy combinational depth of large fixed-point MAC operations required before registering the final result.

```
Starting Points with Worst Slack
********************************

                                                      Starting                                                Arrival
Instance                                              Reference     Type      Pin     Net                     Time        Slack
                                                      Clock
-----------------------------------------------------------------------------------------------------------------------------------
u_layer0.gen_neurons\[9]\.10\.n9\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[6]\.10\.n6\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[0]\.10\.n0\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[2]\.10\.n2\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[3]\.10\.n3\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[5]\.10\.n5\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[7]\.10\.n7\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[8]\.10\.n8\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[4]\.10\.n4\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
u_layer0.gen_neurons\[1]\.10\.n1\.u_n.data_reg_0[0]   nn_top|clk    dffeas    q       data_reg_0[0]           0.845       -1.718
===================================================================================================================================
```

(a) Starting points (`data_reg_0[0]` input pipeline registers).

```
Ending Points with Worst Slack
******************************

                                                      Starting                                                Required
Instance                                              Reference     Type      Pin     Net                     Time        Slack
                                                      Clock
-----------------------------------------------------------------------------------------------------------------------------------
u_layer0.gen_neurons\[5]\.10\.n5\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_28_a_add27 10.264      -1.718
u_layer0.gen_neurons\[6]\.10\.n6\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_33_a_add27 10.264      -1.718
u_layer0.gen_neurons\[7]\.10\.n7\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_38_a_add27 10.264      -1.718
u_layer0.gen_neurons\[3]\.10\.n3\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_18_a_add27 10.264      -1.718
u_layer0.gen_neurons\[8]\.10\.n8\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_43_a_add27 10.264      -1.718
u_layer0.gen_neurons\[4]\.10\.n4\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_23_a_add27 10.264      -1.718
u_layer0.gen_neurons\[0]\.10\.n0\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_3_a_add27  10.264      -1.718
u_layer0.gen_neurons\[1]\.10\.n1\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_8_a_add27  10.264      -1.718
u_layer0.gen_neurons\[2]\.10\.n2\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_13_a_add27 10.264      -1.718
u_layer0.gen_neurons\[9]\.10\.n9\.u_n.product_reg[63] nn_top|clk    dffeas    d       product_reg_next_48_a_add27 10.264      -1.718
===================================================================================================================================
```

(b) Ending points (`product_reg[63]` accumulator registers).

Figure 6: Synthesis timing report showing identical critical paths across parallel `neuron` instances in Layer 0.

2.6 Schematic architecture (RTL)

Answer:

The RTL architecture implements a fully pipelined, streaming 2-layer Deep Neural Network processor. The design is organized hierarchically, with a top-level wrapper managing state control, intermediate buffering, and cascaded dense layers performing the inference computation. The architecture consists of the following key components:

a. **Top-Level Integration**: `nn_top` integrates an input FIFO, a 10-state FSM controller, two instantiated neural network layers (`layer_0`, `layer_1`) buffered by an intermediate FIFO, and a final `argmax` classification module.

b. **Neural Layer**: Each `layer` instantiates a parallel array of `neuron` modules via a `generate-for` loop, integrating their pre-quantized weight arrays and culminating in a ReLU activation function.

c. **Pipelined Neuron MAC**: A module performing the fixed-point multiply-accumulate ($x_i \times w_i + b$) operation with a dedicated DSP-mapped multiplication, shift-based dequantization, and sequential accumulation over the data stream.

Figure 7 illustrates the 10-state FSM control logic. The states sequentially orchestrate the inference lifecycle: beginning at `IDLE` and `FIFO_PREFETCH`, iterating through execution phases (`START`, `RUN`, `WAIT`) for both Layer 0 and Layer 1, and finishing with the `ARGMAX` evaluation before signaling completion in the `DONE` state.

Figure 8 shows the overall system-level integration. The central `statemachine` blocks orchestrate data flow originating from the input interface into `layer_0`. Its sequential outputs are captured into an intermediate `fifo_32s_16s_5s` before evaluating through `layer_1` and feeding the sequential `argmax_10s` classifier.

Figure 9 details the internal logic of a parameterized `layer` module. This module primarily utilizes a hardware `generate` block to instantiate multiple `neuron` copies synchronously operating in parallel, followed by combining logic and a multiplexed ReLU activation boundary.

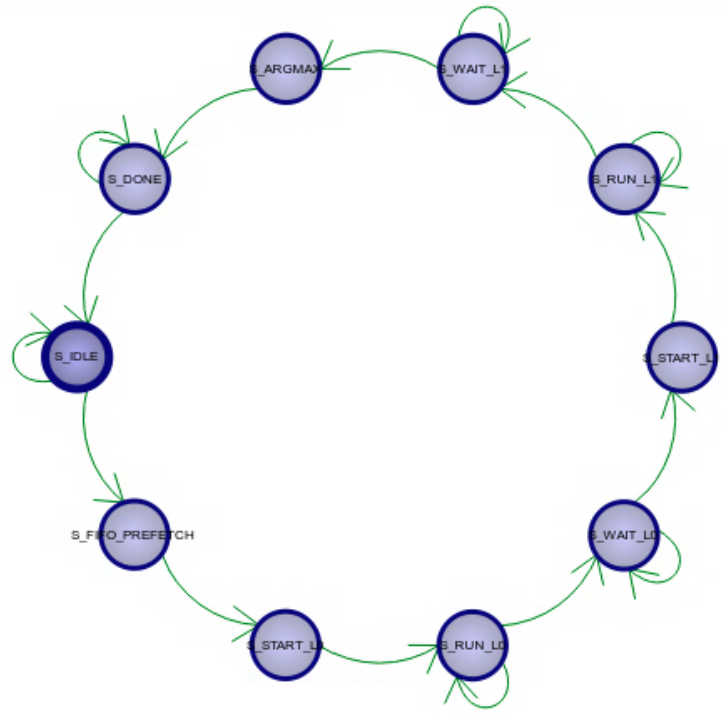


Figure 7: 10-state FSM flow governing the NN inference pipeline: Prefetch → Layer 0 Execution → Layer 1 Execution → Argmax → Done.

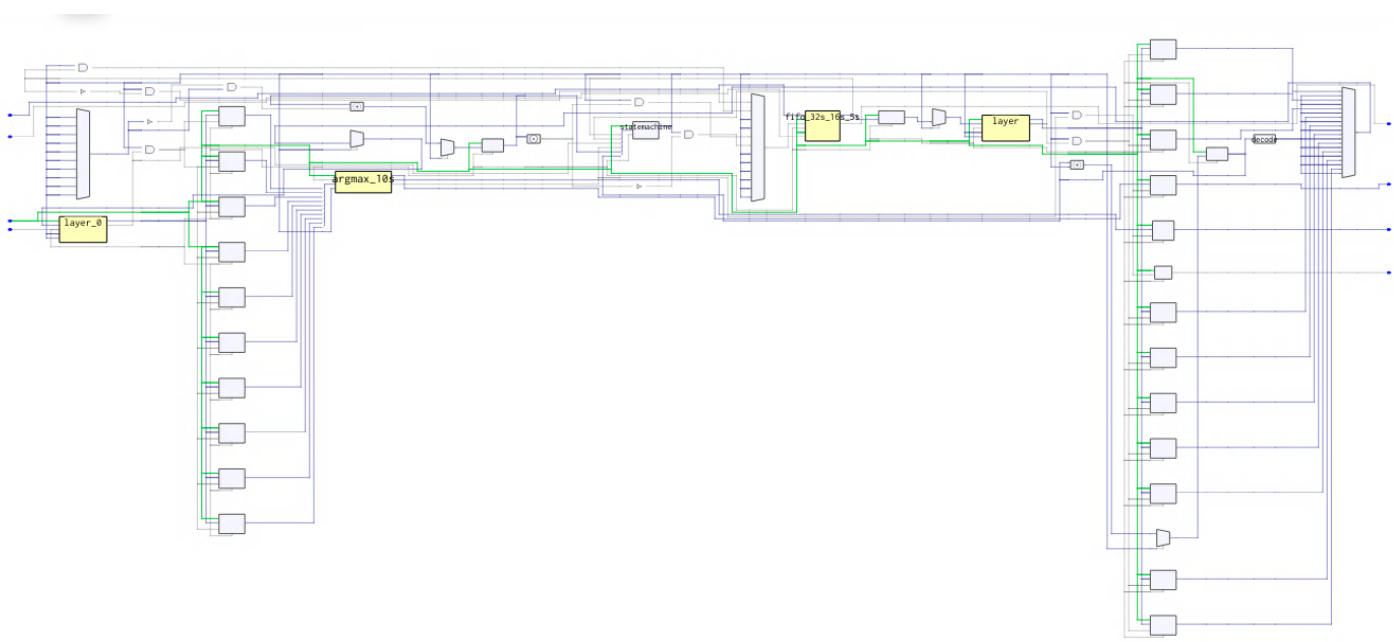


Figure 8: Top-level `nn_top` schematic: Central FSM controlling `layer_0`, intermediate FIFO, `layer_1`, and `argmax_10s` modules.

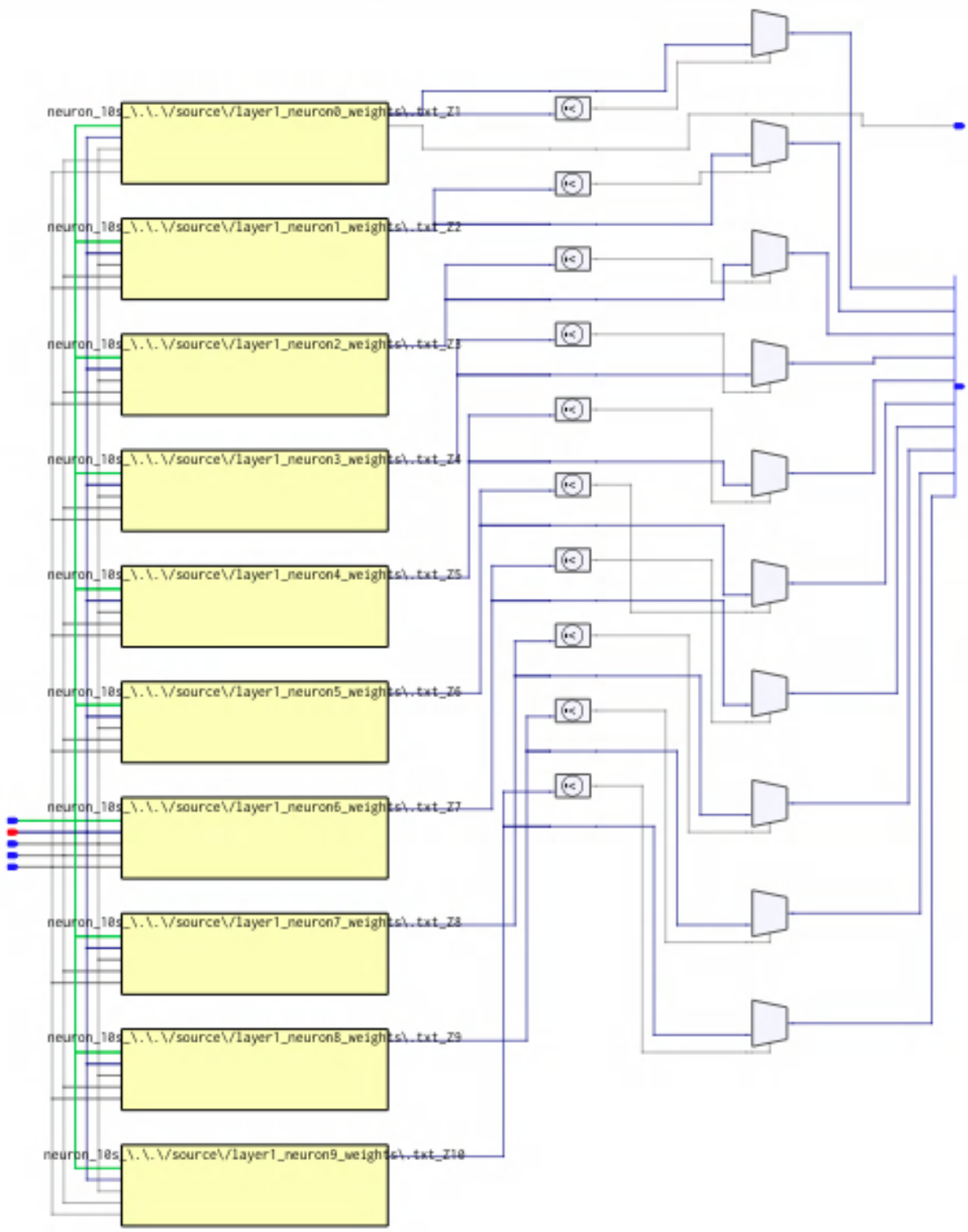


Figure 9: Internal schematic of `layer`: A parallel generated array of pipelined neurons converging into a multiplexed ReLU activation output.

Figure 10 provides a zoomed-in view of the parameterized instantiation of the neurons within Layer 0. Each `neuron_10s` instance is synthesized with its own dedicated memory block, directly initialized by the compiler from unique static hex parameter files (e.g., `layer0_neuron...weights.txt`).

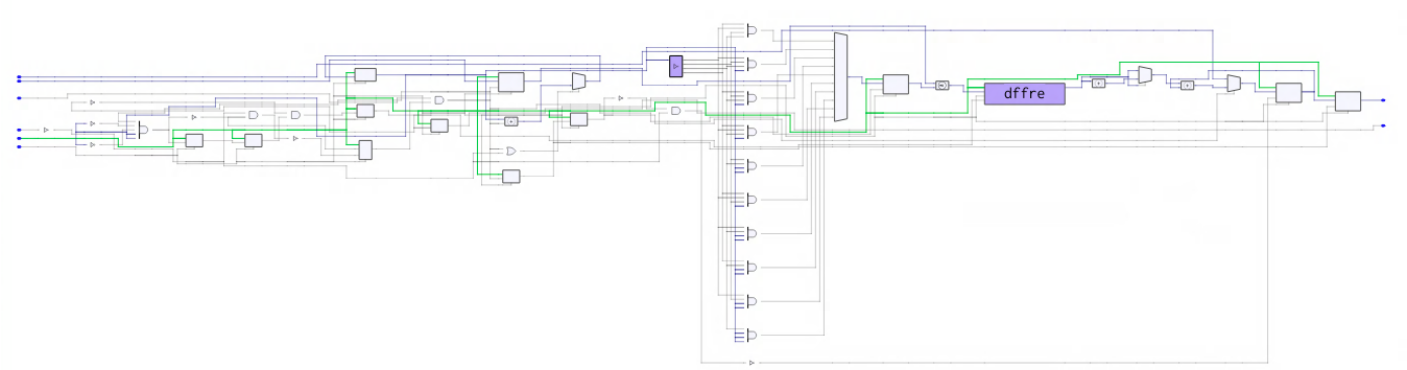


Figure 10: Detailed view of Layer 0 neuron array: 10 parallel `neuron` instances, each uniquely parameterized with discrete weight initialization files.

Figure 11 shows the internal structure of the `argmax` pipeline evaluating probabilities dynamically. The architecture isolates the comparisons into a streamlined iterative flow rather than one gigantic combinational comparator tree:

- **Index & Best Tracking Registers:** Flip-flops (DFFs) sequentially track the active comparison `idx`, the current highest probability `best_val`, and its corresponding label `best_idx`.
- **Streaming Combinational Comparator:** A multiplexer dynamically feeds the array of L1 scores conditionally evaluated alongside the registered best value.
- **Latency Profile:** Computing N inputs iteratively dictates a short deterministic latency sequentially comparing probability variables up until signaling the 10th completion cycle gracefully.

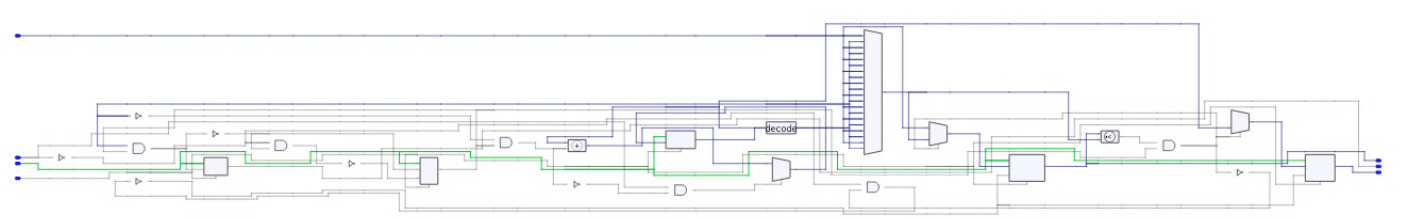


Figure 11: Internal schematic of the `argmax` logic: Iterative DFF tracking and comparison dynamically scaling across the Layer 1 output vectors.

Key Design Choices -

- **Hardware Streaming Pipelining:** Processes inference continuously over 784 sequential cycles to natively limit the immense I/O boundary that fully unrolled layers would otherwise impose.

- **Fixed-Point Precision Matching:** Employs a strict Q14 custom dequantization algorithm replacing generic division operators with synthesizable shift-and-accumulate logic directly matching the C reference's truncation-toward-zero boundaries.
- **Independent Weight Distribution:** Segregating the global parameter matrices into dedicated `.txt` files for every neuron enables the synthesizer to directly map initializations into completely localized dedicated BRAMs.
- **Two-Process FSM Methodology:** Explicit separation of combinational next-state evaluations from sequential state registers ensures glitch-free orchestrations across the heavily pipelined, multi-cycle Layer 0 → Layer 1 execution boundaries.

2.7 Throughput (Calculations-per-second)

Answer:

The hardware streaming architecture achieves highly efficient inference throughput, processing a complete 784-dimension input frame in 818 clock cycles after the initial pipeline startup.

- **Inference Throughput:** At the estimated maximum frequency of 87.3 MHz:

$$\text{Throughput}_{\text{inference}} = \frac{87.3 \text{ MHz}}{818 \text{ cycles/inference}} \approx \mathbf{106.7\ k\ Inferences/s} \tag{1}$$

- **Pixel Processing Throughput:** Each inference evaluates a 28×28 (784 pixels) image over the network continuously:

$$\text{Throughput}_{\text{pixel}} = 106.7 \text{ k Inferences/s} \times 784 \text{ pixels/inference} \approx \mathbf{83.6\ MSamples/s} \tag{2}$$

This capability of evaluating over **106,000 images per second** combined with the pipelined FSM flow enables the processor to be deployed heavily in real-time embedded streaming video.